# BPSC TRE 4.0

## Computer Science (Class 11-12)

# Day 12

## 2. Programming > Algorithms & Complexity

2026-11-08

BPSC TRE 4.0 Computer Science — Day 12 — 2. Programming > Algorithms & Complexity — 2026-11-08

**Notes • 22 Questions • Answer Key + व्याख्या**

# Algorithm क्या है: चित्र




**Algorithm = समस्या हल करने के सीमित, साफ़ steps (भाषा से स्वतंत्र).**

## 5 गुण (properties):

| गुण | मतलब |
|---|---|
| Input | 0 या ज़्यादा inputs |
| Output | कम से कम 1 output |
| Definiteness | हर step साफ़, एक मतलब |
| Finiteness | सीमित steps में रुके |
| Effectiveness | हर step करने लायक/आसान |

## Algorithm vs Program vs Flowchart

| नाम | क्या है |
|---|---|
| Algorithm | steps, किसी भी भाषा में/शब्दों में |
| Program | algorithm का computer language रूप |
| Flowchart | algorithm का चित्र (symbols) |

*TRE 2.0: 'A computer program consists of' -> algorithms written in computer language (option C), flowchart नहीं.*
*Trap: Flowchart चित्र है; program उससे अलग है.*

## Complexity दो तरह की:

Time complexity: steps/समय n के हिसाब से कितना बढ़ता है.
Space complexity: extra memory n के हिसाब से कितनी लगती है.

# Algorithm और Properties

## 1 I. आसान भाषा में

- Algorithm (एल्गोरिदम) = किसी काम को पूरा करने के सीमित और साफ़ steps. जैसे चाय बनाने की विधि.
- Algorithm भाषा से स्वतंत्र है: शब्दों, flowchart या code, किसी में भी लिखो.
- यही steps जब C++ या Python में लिख दें तो program बन जाता है.

## 2 2. रोज़ का उदाहरण

- चाय की recipe: पानी उबालो, पत्ती डालो, दूध डालो, छानो. Input = सामान, steps = विधि, output = चाय. हर step साफ़ और सीमित, यही algorithm है.

## 3 3. पाँच गुण (Diagram: p1)

- Input: 0 या ज़्यादा. Output: कम से कम 1.
- Definiteness: हर step का एक ही मतलब.
- Finiteness: सीमित steps के बाद रुके. Effectiveness: हर step करने लायक.

→ Algorithm, Flowchart और Program में फ़र्क: algorithm = steps, flowchart = उनका चित्र, program = computer language में code.

## 4 4. Exam Point + Trap

- TRE 2.0: "A computer program consists of" -> algorithms written in computer language (C).
- Trap: Flowchart को program समझ लेना. Flowchart सिर्फ चित्र है.

## 5 5. याद रखो

- Algorithm = सीमित साफ़ steps. Program = code में algorithm.

# Growth rate: कौन तेज़ बढ़ता है

| f(n) | n=1 | n=2 | n=4 | n=8 | n=16 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 |
| log n | 0 | 1 | 2 | 3 | 4 |
| n | 1 | 2 | 4 | 8 | 16 |
| n log n | 0 | 2 | 8 | 24 | 64 |
| n^2 | 1 | 4 | 16 | 64 | 256 |
| n^3 | 1 | 8 | 64 | 512 | 4096 |
| 2^n | 2 | 4 | 16 | 256 | 65536 |

**क्रम (छोटा से बड़ा): 1 < log n < n < n log n < n^2 < n^3 < 2^n < n!**

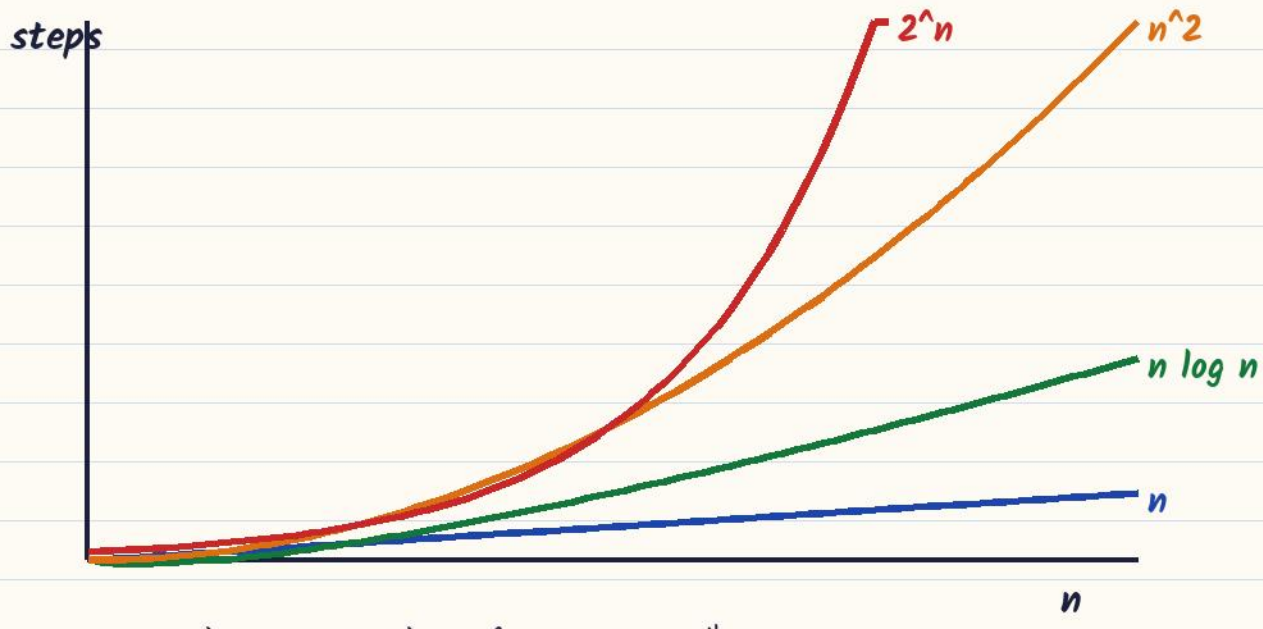
Big-O का नियम: सिर्फ सबसे तेज़ बढ़ने वाला term रखो, constant हटाओ.

3n^2 + 5n + 7 -> O(n^2)

(n log n + n^2)(n^3 + 2) -> n^2 . n^3 = O(n^5) [NET Jun 2013]

(n! + 2^n)(n^3 + log(n^2+1)) -> n! . n^3 = O(n^3 . n!) [option C]

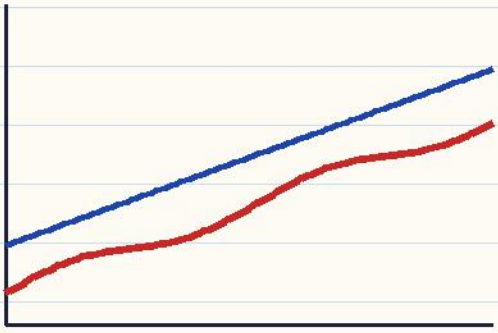
Trap: 2n^2 या 2^n जैसे constant वाले terms अलग से मत लिखो.




n = 0 से 8: 2^n सबसे जल्दी ऊपर भागता है (exponential).

# Big-O, Omega, Theta: चित्र

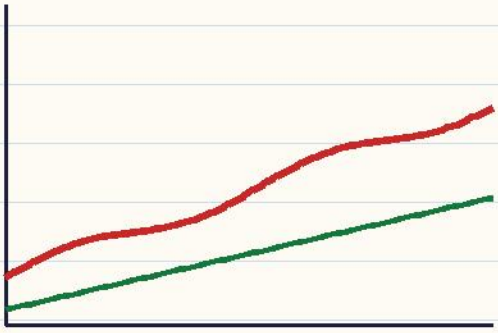
## O (upper bound): f कभी c.g से ऊपर नहीं



लाल = f(n)
c.g(n) ऊपर

O : f(n) <= c.g(n)
(worst-case upper)
Omega: f(n) >= c.g(n)
(best-case lower)
Theta: दोनों एक साथ
(tight bound)

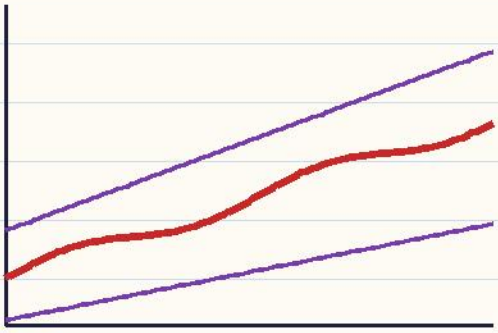
## Omega (lower bound): f कभी c.g से नीचे नहीं



लाल = f(n)
c.g(n) नीचे

TRE 3.0:
Big-O = upper bound
(option B)
lower bound नहीं,
average नहीं.

## Theta (tight): f दोनों के बीच में



लाल = f(n)

little o, little omega:
o(g): सख़्ती से छोटा
omega(g): सख़्ती से बड़ा
o और omega transitive हैं
(NET Dec 2021: o+o => o)

# Complexity: Big-O

## 1. आसान भाषा में

- Complexity बताती है कि input n बढ़ने पर algorithm कितना धीमा होगा (time) या कितनी memory लगेगी (space).
- Notation: Big-O = ऊपरी सीमा (worst), Omega = निचली सीमा (best), Theta = दोनों (tight).

## 2. रोज़ का उदाहरण

- किताब में नाम खोजना: पहले पन्ने से हर पन्ना देखो तो n पन्ने (linear). बीच से खोलकर आधा करते जाओ तो log n (binary search).

## 3. Growth rate का क्रम (Diagram: p2)

- 1 < log n < n < n log n < n^2 < n^3 < 2^n < n!
- Big-O लिखने के नियम: सिर्फ सबसे बड़ा term रखो, constant हटाओ. 3n^2+5n+7 -> O(n^2).
- Product में बड़े terms का गुणा: (n log n + n^2)(n^3 + 2) -> O(n^5).

## 4. O, Omega, Theta (Diagram: p3)

- O: f(n) <= c.g(n). Omega: f(n) >= c.g(n). Theta: दोनों.
- little-o, little-omega: सख़्त (strict) तुलना, transitive.

## 5. Exam Point + Trap

- TRE 3.0: Big-O = upper bound.
- NET: (n!+2^n)(n^3+log(n^2+1)) = O(n^3.n!).
- Trap: Big-O को average या lower bound कहना.

## 6. याद रखो

- बड़ा term रखो, constant हटाओ. O ऊपर, Omega नीचे, Theta बीच.

# Loop गिनो: Time complexity

```
for (i=1; i<=n; i++)
  for (j=1; j<=n; j++)
    count++;
```

बाहर वाला n बार,
अंदर वाला हर बार n बार
कुल = n x n = n^2
-> O(n^2)

| i | j चलता है | count |
|---|---|---|
| 1 | 1..n | n |
| 2 | 1..n | n |
| ... | ... | ... |
| n | 1..n | n |

जोड़: n + n + ... (n बार) = n^2

```
for (i=1; i<=n; i=i*2)
    count++;
```

i = 1,2,4,8,...
k कदम बाद i = 2^k
2^k > n पर रुकता है
k = log2 n -> O(log n)

| n | i के मान | चक्कर |
|---|---|---|
| 8 | 1,2,4,8 | 4 (=log8+1) |
| 16 | 1,2,4,8,16 | 5 |
| 1024 | 1..1024 | 11 |

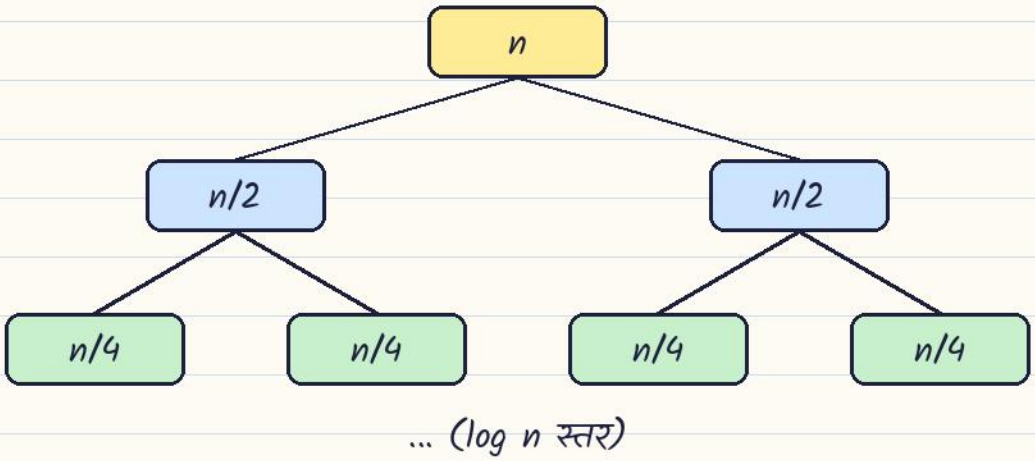
```
for (i=1; i<=n; i++)
  for (j=1; j<=i; j++)
    count++;
```

i=1 पर 1 बार, i=2 पर 2...
1+2+...+n = n(n+1)/2
-> O(n^2)

Trap: दो loops हमेशा n^2 नहीं; अंदर वाला i*=2 हो तो n log n.
Trap: loops अलग-अलग (एक के बाद एक) हों तो जोड़ो: n + n = O(n).
NET Jun 2014: M1 और M2 एक के बाद एक -> h(n)+g(n) = O(max(h,g)).

# Recurrence: Tree और Master

**T(n) = 2T(n/2) + n  (Merge sort)**




**हर स्तर पर कुल काम = n, स्तर = log2 n  =>  T(n) = O(n log n)**

## Master theorem: T(n) = a T(n/b) + n^d

| शर्त | नतीजा | उदाहरण |
|---|---|---|
| d > log_b a | O(n^d) | T=2T(n/2)+n^2 -> n^2 |
| d = log_b a | O(n^d log n) | T=2T(n/2)+n -> n log n |
| d < log_b a | O(n^(log_b a)) | T=8T(n/2)+n^2 -> n^3 |

NET Dec 2013: T(n) = m T(n/2) + n^2

a=m, b=2: log_2 m > 2 जब m > 4 -> O(n^(lg m)) (option B).

T(m) = T(3m/4) + 1: a=1, d=0, log_(4/3) 1 = 0 = d -> theta(lg m).

T(n) = T(n/3)+T(2n/3)+n: हर स्तर पर n, गहराई log_(3/2) n -> n log n.

Trap: Master theorem तभी जब दोनों हिस्से बराबर n/b हों;
T(n/3)+T(2n/3) के लिए recursion tree लगाओ.

**Strassen: 7 sub-multiplications, T=7T(n/2)+n^2 -> n^(log2 7) = n^2.81**

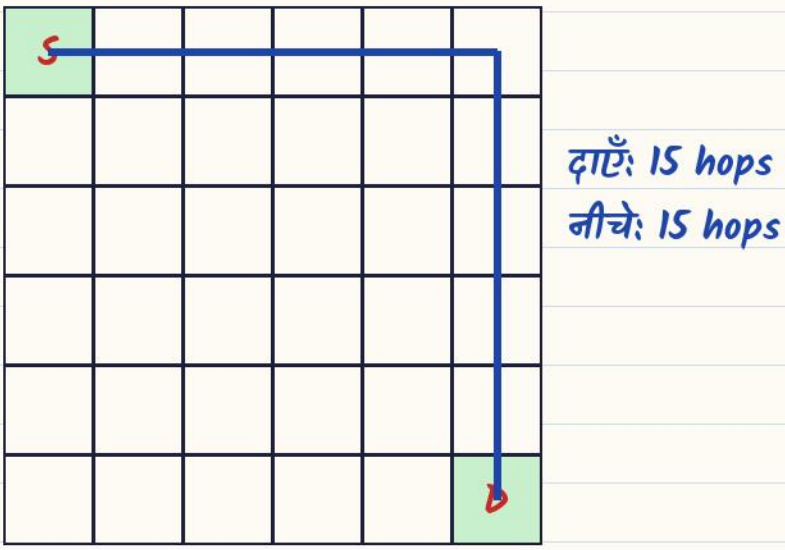
**Divide और Conquer और Combine: तीनों D&C के steps हैं (TRE 3.0).**

# D&C और Grid hops




तीनों D&C के steps हैं; इसलिए 'NOT a step?' का जवाब None of the above (E).

## 16 x 16 grid (256 CPUs): सबसे दूर के दो CPU




(चित्र में 6x6 दिखाया; वही नियम 16x16 पर)

k x k grid: worst = (k-1) + (k-1) = 2(k-1)

16x16: 2 x 15 = 30 hops (option D) [NET Jul 2016]

Trap: 16 या 15 चुनना; दोनों दिशाएँ जोड़ो.

## NP-Complete (TRE 3.0 option C answer):

| समस्या | वर्ग |
|---|---|
| Travelling Salesman | NP-Complete |
| Boolean Satisfiability | NP-Complete |
| Shortest Path | P (poly. में हल) |

# Loop और Recurrence

## 1 I. आसान भाषा में

- Time complexity निकालने के दो तरीके: loops गिनना और recursion का recurrence relation हल करना.

## 2 2. Loops (Diagram: p4)

- एक loop 1..n = O(n). दो nested loops 1..n = O(n^2). अंदर वाला j<=i हो तो 1+2+...+n = n(n+1)/2 = O(n^2).
- i = i2 वाला loop = O(log n). एक के बाद एक loops = जोड़, बड़ा रखो.

```
for (i=1; i<=n; i=i*2)
    count++;   // runs log n times
```

## 3 3. Recurrence और Master theorem (Diagram: p5)

- T(n) = a T(n/b) + n^d. तीन case: d > log_b a -> n^d. d = log_b a -> n^d log n. d < log_b a -> n^(log_b a).
- Merge sort T=2T(n/2)+n -> n log n. Strassen T=7T(n/2)+n^2 -> n^2.81.
- T(n/3)+T(2n/3)+n -> recursion tree, हर स्तर पर n, गहराई log n -> n log n.

## 4 4. Divide and Conquer और NP (Diagram: p6)

- D&C के 3 steps: Divide, Conquer, Combine (TRE 3.0: सब steps हैं, "NOT" का जवाब None).
- P: polynomial में हल (Shortest Path). NP-Complete: TSP, SAT (TRE 3.0).
- Grid में worst delay = 2(k-1); 16x16 में 30 hops.

## 5 5. Exam Point + Trap

- Trap: दो loops को हमेशा n^2 मानना. Trap: Master theorem unequal हिस्सों पर लगाना.

- Trap: binary search में log n और log n + 1 में गड़बड़.

### 6 6. याद रखो

- Loop गिनो, recurrence में a, b, d पहचानो, फिर Master.

## BPSC ऐसे पूछता है

### 1 1. TRE में आए प्रकार

- Definition: "A computer program consists of" (TRE 2.0) -> algorithms in a computer language.
- Statement true: "Big-O notation" (TRE 3.0) -> upper bound.
- "NOT" type: "NOT a step in Divide and Conquer" (TRE 3.0) -> तीनों steps हैं, इसलिए None of the above (E).
- "NOT NP-Complete" (TRE 3.0) -> Shortest Path.
- TRE में 5 options होते हैं और "More than one"/"None of the above" आम हैं; हर option अलग जाँचो.

### 2 2. याद रखो

- Definition, bound का नाम और NP की पहचान, यही TRE के सवाल हैं.

## NET level में ऐसे आता है

### 1 1. प्रकार

- Big-O estimate: (n log n+n^2)(n^3+2) -> n^5.
- Recurrence: T(m)=T(3m/4)+1 -> theta(lg m); T(n)=mT(n/2)+n^2.
- Known results: Strassen n^2.81, LIS n lg n, RB-tree lg n, m-coloring O(n m^n).
- Asymptotic relations (o, omega transitive), 2 modules h(n)+g(n).

- Grid delay (hops).

### ② 2. याद रखो

- पहले type पहचानो, फिर मिले result याद करो या Master लगाओ.

## Numericals — और practice

### ① Practice 1: loop गिनती

- Q: for(i=1;i<=n;i=i3) में n=81 पर कितने चक्कर? Solution: i=1,3,9,27,81 = 5 चक्कर = log3 81 + 1. Answer 5.
- Q2: for(i=1;i<=n;i++) for(j=1;j<=n;j=j2) की complexity? Solution: बाहर n, अंदर log n, कुल n log n.

### ② Practice 2: Master theorem

- Q: T(n)=2T(n/2)+n^2. a=2,b=2,d=2, log2 2 = 1 < 2 -> O(n^2).
- Q2: T(n)=8T(n/2)+n. log2 8 = 3 > 1 -> O(n^3).

### ③ Practice 3: Big-O

- Q: (n^2+n)(log n+5) -> n^2 log n.
- Q2: 5n^3+2^n+n! -> n!.

### ④ Practice 4: Grid

- Q: 8x8 grid में worst hops = 2 x 7 = 14.
- Q2: 10x10 में 2 x 9 = 18.

## Abbreviations

### ① Full Forms

- TRE: Teacher Recruitment Examination
- NET: National Eligibility Test
- BPSC: Bihar Public Service Commission

- P: Polynomial time
- NP: Non-deterministic Polynomial time
- SAT: Boolean Satisfiability
- TSP: Travelling Salesman Problem
- D&C: Divide and Conquer
- LIS: Longest Increasing Subsequence
- DP: Dynamic Programming
- C++: C Plus Plus (programming language)
- CPU: Central Processing Unit

# Questions + हर सवाल के नीचे Explanation

**Q1.** [BPSC TRE 2.0 (2023)]

**A computer program consists of**

(A) system flowchart
(B) program flowchart
(C) algorithms written in computer language
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: system flowchart पूरे system का चित्र है
(B) गलत: program flowchart चित्र है, program नहीं
(C) ✔ सही: program असल में computer language में लिखे algorithm हैं
(D) गलत: एक ही सही है
(E) गलत: एक option सही है

**Ans: (C) algorithms written in computer language**

Algorithm steps हैं. जब वे steps किसी computer language (C++, Python) में लिखे जाते हैं तो वही program बन जाता है. Flowchart सिर्फ चित्र है. जवाब (C). Diagram: p1.

---

**Q2.** [BPSC TRE 3.0 (2024)]

**Which of the following statements is true about Big-O notation?**

(A) It represents the lower bound of an algorithm’s runtime.
(B) It represents the upper bound of an algorithm’s runtime
(C) It represents the average runtime of an algorithm.
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: lower bound Omega होता है
(B) ✔ सही: Big-O ऊपरी सीमा (upper bound) देता है
(C) गलत: average case Theta या औसत विश्लेषण से आता है
(D) गलत: सिर्फ B सही है
(E) गलत: B सही है

**Ans: (B) It represents the upper bound of an algorithm’s runtime**

Big-O बताता है कि algorithm इससे ज़्यादा धीमा नहीं होगा, यानी upper bound. Omega = lower bound, Theta = tight bound. जवाब (B). Diagram: p3.

---

**Q3.** [BPSC TRE 3.0 (2024)]

**Which of the following is NOT a step in the Divide and Conquer algorithm?**

(A) Combine
(B) Conquer
(C) Divide
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: Combine D&C का आख़िरी step है
(B) गलत: Conquer (हर हिस्सा हल करना) भी step है
(C) गलत: Divide पहला step है
(D) गलत: तीनों step हैं, इसलिए "एक से ज़्यादा" नहीं
(E) ✔ सही: तीनों (Divide, Conquer, Combine) D&C के steps हैं, इसलिए None of the above

**Ans: (E) None of the above**

Divide and Conquer: Divide (बाँटो), Conquer (हर हिस्सा हल करो), Combine (जवाब जोड़ो). तीनों steps हैं, इसलिए "NOT" वाला कोई नहीं. जवाब (E). Diagram: p6.

---

**Q4.** [BPSC TRE 3.0 (2024)]

**Which of the following is NOT an NP-Complete problem?**

(A) Traveling Salesman Problem
(B) Boolean Satisfiability Problem
(C) Shortest Path Problem
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) गलत: TSP का decision रूप NP-Complete है
(B) गलत: SAT पहली NP-Complete समस्या है (Cook)
(C) ✔ सही: Shortest Path (Dijkstra) polynomial समय में हल होती है, यह P में है
(D) गलत: सिर्फ C सही है
(E) गलत: C सही है

**Ans: (C) Shortest Path Problem**

Shortest path को Dijkstra O(V^2) या O(E log V) में हल करता है, इसलिए NP-Complete नहीं. TSP और SAT NP-Complete हैं. जवाब (C). Diagram: p6.

---

**Q5.** [UGC NET Jun 2012 (Paper 3)]

**The upper bound of computing time of m coloring decision problem is**

(A) O(nm)
(B) O(n^m)
(C) O(n m^n)
(D) O(n^m m^n)

हर **option** समझो:

(A) गलत: nm सिर्फ दो संख्याओं का गुणा है, सारे रंग-मेल नहीं
(B) गलत: n^m उलटा है, बदलना m^n चाहिए
(C) ✔ सही: n nodes को m रंगों में m^n तरीकों से रंग सकते हैं, हर जाँच में n काम: O(n m^n)
(D) गलत: n^m m^n जरूरत से बहुत बड़ा है

**Ans: (C) O(n m^n)**

m-coloring में हर node के लिए m रंग चुनने हैं, यानी कुल m^n संभावनाएँ. हर संभावना जाँचने में लगभग n काम. इसलिए upper bound O(n m^n). जवाब (C).

---

**Q6.** [UGC NET Jun 2013]

**Give as good a big-O estimate as possible for the following functions: (nlogn+n2) (n3+2) and (n! +2n) (n3+log(n2+1))**

(A) O(n^5+2n^2) & O(n^3*n!)
(B) O(n^5) & O(n^3*2^n)
(C) O(n^5) & O(n^3*n!)
(D) O(n^5+2n^2) & O(n^3*2^n)

हर **option** समझो:

(A) गलत: दूसरा हिस्सा n^3 x n! है, 2n नहीं, और 2n^2 अलग से नहीं लिखते
(B) गलत: n! 2^n से बहुत तेज़ बढ़ता है
(C) ✔ सही: पहला n^5, दूसरा n^3.n!
(D) गलत: दोनों हिस्से गलत

**Ans: (C) O(n^5) & O(n^3*n!)**

पहला: (n log n + n^2) में n^2 बड़ा, (n^3+2) में n^3 बड़ा, गुणा n^5. दूसरा: (n!+2^n) में n! बड़ा, (n^3 + log) में n^3 बड़ा, गुणा n^3.n!. जवाब (C). Diagram: p2.

**Step-by-step Solution:**

1. हर bracket में सबसे तेज़ बढ़ने वाला term रखो.
2. पहला: n^2 x n^3 = n^5.
3. दूसरा: n! x n^3.
4. Answer (C). Trap: 2n^2 जैसे छोटे terms साथ में लिख देना.

**Q7.** [UGC NET Sep 2013 (Paper 3)]

**The time complexity of recurrence relation**
**T(n) = T(n/3) + T(2n/3) + O(n) is**

(A) O(lg n)
(B) O(n)
(C) O(n lg n)
(D) O(n^2)

हर **option** समझो:

(A) गलत: lg n तब आता जब हर स्तर पर constant काम हो
(B) गलत: हर स्तर पर n काम है और स्तर कई हैं, सिर्फ n नहीं
(C) ✔ सही: हर स्तर पर कुल n, गहराई log n -> n log n
(D) गलत: n^2 बहुत ज़्यादा है

**Ans: (C) O(n lg n)**

T(n/3)+T(2n/3)+n: tree के हर स्तर पर हिस्सों का जोड़ n ही रहता है. सबसे लंबी शाखा की गहराई log_{3/2} n. कुल n log n. जवाब (C). Diagram: p5.

**Step-by-step Solution:**

1. Tree बनाओ: root n; बच्चे n/3 और 2n/3; जोड़ = n.
2. हर स्तर का जोड़ = n.
3. गहराई = log_(3/2) n = O(log n).
4. कुल = n x log n. Answer (C).

---

**Q8.** [UGC NET Sep 2013 (Paper 3)]

**The number of possible parenthesizations of a sequence of n matrices is**

(A) O(n)
(B) θ(n lg n)
(C) Ω(2^n)
(D) None of the above

हर **option** समझो:

(A) गलत: n parenthesizations बहुत कम हैं
(B) गलत: n lg n भी कम है
(C) ✔ सही: संख्या Catalan number है, जो 2^n से भी तेज़ बढ़ती है, इसलिए Omega(2^n)
(D) गलत: C सही है

**Ans: (C) Ω(2^n)**

n matrices के parenthesization की संख्या Catalan number C(n-1) होती है, जो exponential है (लगभग 4^n). इसलिए Omega(2^n). जवाब (C).

**Q9.** [UGC NET Sep 2013 (Paper 3)]

**The time complexity of an efficient algorithm to find the longest monotonically increasing subsequence of n numbers is**

(A) O(n)
(B) O(n lg n)
(C) O(n^2)
(D) None of the above

हर **option** समझो:

(A) गलत: O(n) में सबसे लंबा बढ़ता subsequence नहीं निकलता
(B) ✔ सही: binary search वाली तरकीब से O(n lg n)
(C) गलत: O(n^2) सीधा DP तरीका है, efficient नहीं
(D) गलत: B सही है, इसलिए None नहीं

**Ans: (B) O(n lg n)**

Longest increasing subsequence का सीधा DP O(n^2) है, पर patience-sorting/binary search से O(n lg n). इसलिए efficient algorithm O(n lg n). जवाब (B).

---

**Q10.** [UGC NET Dec 2013 (Paper 3)]

**Let A and B be two n × n matrices.**
**The efficient algorithm to multiply the two matrices has the time complexity**

(A) O(n^3)
(B) O(n^2.81)
(C) O(n^2.67)
(D) O(n^2)

हर **option** समझो:

(A) गलत: O(n^3) साधारण तरीका है
(B) ✔ सही: Strassen का तरीका O(n^2.81)
(C) गलत: n^2.67 अभी तक के Strassen में नहीं
(D) गलत: n^2 से कम में अभी तक कोई तरीका नहीं मिला

**Ans: (B) O(n^2.81)**

साधारण matrix गुणा O(n^3). Strassen 7 sub-multiplications से O(n^(log2 7)) = O(n^2.81). जवाब (B). Diagram: p5.

---

**Q11.** [UGC NET Dec 2013 (Paper 3)]

**The recurrence relation T(n) = m T(n/2) + a n^2 is satisfied by**

(A) O(n^2)
(B) O(n^(lg m))
(C) O(n^2 lg n)
(D) O(n lg n)

हर **option** समझो:

(A) गलत: n^2 तब जब m < 4
(B) ✔ सही: m > 4 पर O(n^(lg m)) (key)
(C) गलत: n^2 lg n तब जब m = 4
(D) गलत: n lg n तब जब m = 2 और काम n हो, यहाँ काम n^2 है

**Ans: (B) O(n^(lg m))**

T(n) = m T(n/2) + n^2. Master theorem: a=m, b=2, d=2. log_2 m > 2 यानी m > 4 पर जवाब n^(lg m); m=4 पर n^2 lg n; m<4 पर n^2. Key के अनुसार (B). Trap: m की कीमत देखे बिना जवाब चुनना.

**Step-by-step Solution:**

1. a=m, b=2, d=2; log_2 m की तुलना 2 से.
2. m>4: n^(lg m); m=4: n^2 lg n; m<4: n^2.
3. Key (B) m > 4 वाला case. Diagram: p5 Master table.

---

**Q12.** [UGC NET Jun 2014]

**An algorithm is made up of 2 modules M1 and M2. If time complexity of modules M1 and M2 are h(n) and g(n) respectively, the time complexity of the algorithm is**

(A) min (h(n), g(n))
(B) max (h(n), g(n))
(C) h(n) + g(n)
(D) h(n) * g(n)

हर **option** समझो:

(A) गलत: min लेने से बुरा case नहीं मिलता
(B) ✔ सही: दोनों modules एक के बाद एक चलते हैं, कुल h+g, जो O(max(h,g)) होता है
(C) गलत: जोड़ है, गुणा नहीं
(D) गलत: गुणा तब जब M1 के अंदर M2 चले

**Ans: (B) max (h(n), g(n))**

Big-O में बड़ा term जीतता है, इसलिए एक के बाद एक चलने वाले modules की complexity max होती है (key (B)); जोड़ भी सही सोच है पर O(max) कहलाता है. जवाब (B). Diagram: p4.

---

**Q13.** [UGC NET Jul 2016 (Paper 3)]

**A multicomputer with 256 CPUs is organized as 16 × 16 grid. What is the worst case delay (in hops) that a message might have to take ?**

(A) 16
(B) 15
(C) 32
(D) 30

हर **option** समझो:

(A) गलत: 16 सिर्फ एक दिशा से थोड़ा ज़्यादा है
(B) गलत: 15 सिर्फ एक दिशा के hops हैं
(C) गलत: 32 ज़्यादा है, 16x16 में सबसे दूर 15+15
(D) ✔ सही: कोने से कोने तक 15 + 15 = 30 hops

**Ans: (D) 30**

Grid में message दाएँ और नीचे चलता है. सबसे दूर के कोने: 15 hops एक दिशा, 15 दूसरी. कुल 30. जवाब (D). Diagram: p6.

**Step-by-step Solution:**

1. k x k grid में worst = 2(k-1).
2. k=16: 2 x 15 = 30 hops.
3. Answer (D). Trap: 16 या 15 चुनना.

---

**Q14.** [UGC NET Jan 2017 (Paper 3)]

**Red-black trees are one of many search tree schemes that are “balanced” in order to guarantee that basic dynamic-set operations take**

(A) O(1)
(B) O(lg n)
(C) O(n)
(D) O(n lg n)

हर **option** समझो:

(A) गलत: O(1) में search नहीं होता
(B) ✔ सही: Red-black tree की ऊँचाई O(lg n), इसलिए search/insert/delete O(lg n)
(C) गलत: O(n) तो साधारण BST के worst case में है
(D) गलत: n lg n ज़रूरत से ज़्यादा

**Ans: (B) O(lg n)**

Red-black tree balanced है; ऊँचाई 2 lg(n+1) से ज़्यादा नहीं. इसलिए search, insert, delete सब O(lg n). जवाब (B).

---

**Q15.** [UGC NET Jun 2018]

**The solution of the recurrence relation T(m) = T(3m/4) + 1 is :**

(A) θ(lg m)
(B) θ(m)
(C) θ(m lg m)
(D) θ(lg lg m)

हर **option** समझो:

(A) ✔ सही: हर बार m का 3/4 बचता है, गहराई log m, हर स्तर 1 काम
(B) गलत: theta(m) तब जब काम m हो
(C) गलत: m lg m तब जब काम m हो
(D) गलत: lg lg m तब जब m वर्गमूल से घटे

**Ans: (A) θ(lg m)**

T(m) = T(3m/4) + 1. Master: a=1, b=4/3, d=0, log_b a = 0 = d, इसलिए theta(lg m). जवाब (A). Diagram: p5.

**Step-by-step Solution:**

1. a=1, b=4/3, d=0.
2. log_(4/3) 1 = 0 = d -> theta(n^0 log n).
3. Answer theta(lg m) (A). Trap: lg lg m चुनना.

---

**Q16.** [UGC NET Dec 2021-Jun 2022]

**Assume that f(n) and g(n) are asymptotically positive. Which of the following is correct?**

(A) f(n)=O(g(n)) and g(n)=O(h(n))⇨ f(n)=ω(h(n))
(B) f(n)=Ω(g(n)) and g(n)=Ω(h(n))⇨ f(n)=O(h(n))
(C) f(n)=o(g(n)) and g(n)=o(h(n))⇨ f(n)=o(h(n))
(D) f(n)=ω(g(n)) and g(n)=ω(h(n))⇨ f(n)=Ω(h(n))

हर **option** समझो:

(A) गलत: O और O से omega नहीं निकलता
(B) गलत: Omega और Omega से O नहीं निकलता
(C) ✔ सही (key): o का transitive नियम सही है
(D) गलत/सही: omega transitive है, इसलिए यह भी सही दिखता है; key (C)

**Ans: (C) f(n)=o(g(n)) and g(n)=o(h(n))⇨ f(n)=o(h(n))**

Little-o और little-omega दोनों transitive हैं. (A) और (B) साफ़ गलत हैं. Key (C). Logic से (D) भी सही लगता है (omega, Omega की ही तरह transitive), पर exam की key (C) है. Diagram: p3.

---

**Q17.** [Practice]

```
for (i = 1; i <= n; i = i*2) count++;  For n = 64, how many times does count++ run?
```

(A) 5
(B) 6
(C) 7
(D) 8

हर **option** समझो:

(A) गलत: i=1,2,4,8,16,32 तक 6 ही गिने, 64 छूट गया
(B) गलत: 6 में आख़िरी चक्कर i=64 नहीं गिना
(C) ✔ सही: i = 1,2,4,8,16,32,64 यानी 7 बार
(D) गलत: 8 में i=128 भी गिन लिया, जो शर्त तोड़ता है

**Ans: (C) 7**

i हर बार दुगुना होता है और 64 तक शर्त सही है. मान: 1,2,4,8,16,32,64 = 7. यानी log2 64 + 1 = 7. जवाब (C).

**Step-by-step Solution:**

1. i के मान लिखो: 1,2,4,8,16,32,64.
2. i=128 > 64, loop रुक गया.
3. गिनती = 7 = log2(64) + 1.
4. Answer (C). Trap: log2 64 = 6 पर ही रुक जाना.

---

**Q18.** [Practice]

**The solution of T(n) = 4T(n/2) + n is**

(A) O(n)
(B) O(n lg n)
(C) O(n^2)
(D) O(n^3)

हर **option** समझो:

(A) गलत: n सिर्फ combine का काम है, पूरा नहीं
(B) गलत: n lg n तब जब a=2
(C) ✔ सही: a=4,b=2,d=1; log2 4 = 2 > 1, इसलिए n^2
(D) गलत: n^3 तब जब a=8

**Ans: (C) O(n^2)**

Master theorem: a=4, b=2, d=1. log_b a = 2, d से बड़ा, इसलिए O(n^(log_b a)) = O(n^2). जवाब (C).

**Step-by-step Solution:**

1. a=4, b=2, f(n)=n -> d=1.
2. log_2 4 = 2.
3. d (1) < 2 -> जवाब n^(log_b a) = n^2.
4. Answer (C).

**Q19.** [Practice]

```
for (i=1;i<=n;i++) for (j=1;j<=i;j++) count++;  For n = 10, final value of count is
```

(A) 45
(B) 50
(C) 55
(D) 100

हर **option** समझो:

(A) गलत: 45 = 1+...+9, i=10 छूट गया
(B) गलत: 50 का कोई आधार नहीं
(C) ✔ सही: 1+2+...+10 = 10 x 11 / 2 = 55
(D) गलत: 100 तब जब अंदर का loop भी n तक जाए

**Ans: (C) 55**

अंदर का loop i बार चलता है. i=1..10 पर 1+2+...+10 = n(n+1)/2 = 55. जवाब (C). Diagram: p4.

**Step-by-step Solution:**

1. i=1 पर 1, i=2 पर 2, ... i=10 पर 10.
2. जोड़ = n(n+1)/2 = 10 x 11 / 2.
3. = 55. Answer (C). Trap: 100 (n x n) चुनना.

**Q20.** [Practice]

**Which has the SMALLEST growth rate for large n?**

(A) n log n
(B) n^2
(C) 2^n
(D) log n

हर **option** समझो:

(A) गलत: n log n, log n से बड़ा है
(B) गलत: n^2 और बड़ा है
(C) गलत: 2^n सबसे बड़ा है
(D) ✔ सही: क्रम 1 < log n < n < n log n < n^2 < 2^n, इसलिए log n सबसे छोटा

**Ans: (D) log n**

Growth rate का क्रम याद रखो: 1 < log n < n < n log n < n^2 < n^3 < 2^n < n!. दिए चारों में log n सबसे छोटा. जवाब (D). Diagram: p2.

**Q21.** [Practice]

**f(n) = 3n^2 + 5n + 7 is**

(A) θ(n)
(B) θ(n^2)
(C) θ(n^3)
(D) θ(2^n)

हर **option** समझो:

(A) गलत: n से तेज़ बढ़ता है
(B) ✔ सही: सबसे बड़ा term n^2 है, constant 3 हटता है
(C) गलत: n^3 ऊपर की सीमा तो है, tight नहीं
(D) गलत: 2^n बहुत बड़ा

**Ans: (B) θ(n^2)**

Big-O/Theta में सबसे बड़ा term रखो और constant हटाओ: 3n^2 -> n^2. n^2 ऊपर और नीचे दोनों से बाँधता है, इसलिए θ(n^2). जवाब (B). Diagram: p3.

---

**Q22.** [Practice]

**Binary search on a sorted array of 1024 elements needs at most how many comparisons?**

(A) 10
(B) 11
(C) 512
(D) 1024

हर **option** समझो:

(A) गलत: log2 1024 = 10 है, पर आख़िरी बचा element भी जाँचना पड़ता है
(B) ✔ सही: floor(log2 n) + 1 = 11
(C) गलत: 512 तो एक बार आधा करने के बाद बचा
(D) गलत: 1024 linear search का worst case है

**Ans: (B) 11**

हर तुलना में array आधी: 1024 -> 512 -> ... -> 1, यानी 10 बार आधी. आख़िरी element की जाँच जोड़कर 11. जवाब (B).

**Step-by-step Solution:**

1. n=1024. हर कदम पर आधा: 1024,512,256,128,64,32,16,8,4,2,1.
2. 1 तक आने में 10 बार आधा, और हर कदम पर 1 तुलना, अंत में 1 और.
3. कुल = log2 1024 + 1 = 11.
4. Answer (B). Trap: 10 चुनना.

---

# Answer Key

| | | | | |
|---|---|---|---|---|
| **Q1 - C** | **Q2 - B** | **Q3 - E** | **Q4 - C** | **Q5 - C** |
| **Q6 - C** | **Q7 - C** | **Q8 - C** | **Q9 - B** | **Q10 - B** |
| **Q11 - B** | **Q12 - B** | **Q13 - D** | **Q14 - B** | **Q15 - A** |
| **Q16 - C** | **Q17 - C** | **Q18 - C** | **Q19 - C** | **Q20 - D** |
| **Q21 - B** | **Q22 - B** | | | |